

EXPERIMENT – 07

Date: 01/10/2026

Demonstrate 10 Exception Classes in Java with Code Examples

1. AIM / OBJECTIVE

To study and demonstrate different exception classes in Java and understand how exceptions can be handled using try, catch, finally, throw, and throws.

2. PROBLEM STATEMENT

Write a Java program to demonstrate 10 different exception classes in Java with suitable code examples and handle each exception using appropriate exception-handling mechanisms.

3. INTRODUCTION TO EXCEPTION

An **exception** is an unexpected problem that occurs during program execution and interrupts the normal flow of the program.

Example: `int a = 10; int b = 0; System.out.println(a / b);`

Trying to divide 10 by 0 produces an `ArithmeticException`. Exception handling prevents the program from stopping abnormally.

4. EXCEPTION HANDLING

try: Contains code that may cause an exception.

catch: Handles the exception.

finally: Executes whether an exception occurs or not.

throw: Used to explicitly throw an exception.

throws: Used to declare that a method may throw an exception.

5. TYPES OF EXCEPTIONS

Java exceptions can broadly be classified into checked and unchecked exceptions.

Checked Exception: Checked by the compiler. Example: `FileNotFoundException`, `IOException`.

Unchecked Exception: Occurs at runtime. Examples: `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException`, `NumberFormatException`.

6. TEN EXCEPTION CLASSES

No.	Exception	Common Cause
1	<code>ArithmeticException</code>	Division by zero
2	<code>ArrayIndexOutOfBoundsException</code>	Invalid array index
3	<code>StringIndexOutOfBoundsException</code>	Invalid String index
4	<code>NullPointerException</code>	Using a null reference
5	<code>NumberFormatException</code>	Invalid String-to-number conversion
6	<code>IllegalArgumentException</code>	Invalid argument passed to a method
7	<code>ClassCastException</code>	Invalid type casting
8	<code>InputMismatchException</code>	Wrong input type
9	<code>NegativeArraySizeException</code>	Creating array with negative size
10	<code>FileNotFoundException</code>	File does not exist / cannot be opened

7. EXCEPTION 1 – ArithmeticException

Occurs when an invalid arithmetic operation is performed, commonly division by zero.

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
```

8. EXCEPTION 2 – ArrayIndexOutOfBoundsException

Occurs when an array is accessed using an invalid index.

```
try {
    int[] numbers = {10, 20, 30};
    System.out.println(numbers[5]);
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Invalid array index");
}
```

9. EXCEPTION 3 – StringIndexOutOfBoundsException

Occurs when a String is accessed using an invalid index.

```
try {
    String name = "Java";
    System.out.println(name.charAt(10));
} catch (StringIndexOutOfBoundsException e) {
    System.out.println("Invalid String index");
}
```

10. EXCEPTION 4 – NullPointerException

Occurs when an object reference whose value is null is used.

```
try {
    String name = null;
    System.out.println(name.length());
} catch (NullPointerException e) {
    System.out.println("Object reference is null");
}
```

11. EXCEPTION 5 – NumberFormatException

Occurs when an invalid String is converted into a number.

```
try {
    String value = "ABC";
    int number = Integer.parseInt(value);
} catch (NumberFormatException e) {
    System.out.println("Invalid number format");
}
```

12. EXCEPTION 6 – IllegalArgumentException

Occurs when an inappropriate or invalid argument is passed to a method.

```
try {
    Thread.sleep(-100);
} catch (IllegalArgumentException e) {
    System.out.println("Invalid argument");
} catch (InterruptedException e) {
    System.out.println("Thread interrupted");
}
```

13. EXCEPTION 7 – ClassCastException

Occurs when an object is cast to an incompatible class.

```
try {
    Object value = "Java";
    Integer number = (Integer) value;
} catch (ClassCastException e) {
    System.out.println("Invalid type casting");
}
```

14. EXCEPTION 8 – InputMismatchException

Occurs when the input type does not match the type expected by Scanner.

```
try {
    Scanner sc = new Scanner("ABC");
}
```

```

        int age = sc.nextInt();
    } catch (InputMismatchException e) {
        System.out.println("Please enter a valid integer");
    }
}

```

15. EXCEPTION 9 – NegativeArraySizeException

Occurs when an array is created with a negative size.

```

try {
    int size = -5;
    int[] numbers = new int[size];
} catch (NegativeArraySizeException e) {
    System.out.println("Array size cannot be negative");
}

```

16. EXCEPTION 10 – FileNotFoundException

Occurs when Java tries to open a file that does not exist or cannot be opened.

```

try {
    FileReader file = new FileReader("abc.txt");
} catch (FileNotFoundException e) {
    System.out.println("File not found");
}

```

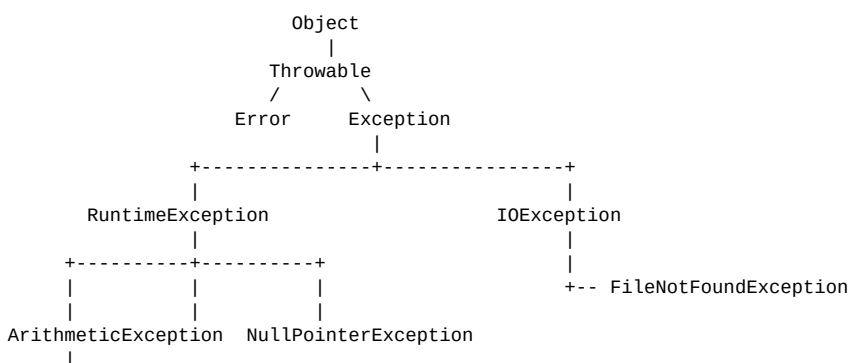
17. ALGORITHM / LOGIC

1. Start the program.
2. Import java.util.* and java.io.*.
3. Create separate try-catch blocks for the 10 exception classes.
4. Perform operations that cause each exception.
5. Handle every exception using its appropriate catch block.
6. Display suitable messages.
7. Stop the program.

18. PROCEDURE

1. Create a Java class named TenExceptionsDemo.
2. Import java.util.* and java.io.*.
3. Write separate try-catch blocks for each exception.
4. Perform an operation that generates ArithmeticException.
5. Access an invalid array index.
6. Access an invalid String position.
7. Use a null reference.
8. Convert an invalid String into an integer.
9. Pass an invalid argument.
10. Perform invalid type casting.
11. Provide an incorrect input type.
12. Create an array with a negative size.
13. Try to open a non-existing file.
14. Handle every exception using its appropriate catch block.
15. Display suitable error messages.
16. Compile and execute the program.
17. Verify that the program completes normally.

19. UML / EXCEPTION HIERARCHY DIAGRAM



```

+-- ArrayIndexOutOfBoundsException
+-- StringIndexOutOfBoundsException
+-- NumberFormatException
+-- IllegalArgumentException
+-- ClassCastException
+-- NegativeArraySizeException
|
+-- NoSuchElementException
|
+-- InputMismatchException

```

Note: InputMismatchException extends NoSuchElementException.

20. COMPLETE JAVA PROGRAM

File Name: TenExceptionsDemo.java

```

import java.util.*;
import java.io.*;

public class TenExceptionsDemo {

    public static void main(String[] args) {

        // 1. ArithmeticException
        System.out.println("==== 1. ArithmeticException =====");

        try {
            int a = 10;
            int b = 0;
            int result = a / b;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        }

        // 2. ArrayIndexOutOfBoundsException
        System.out.println("\n==== 2. ArrayIndexOutOfBoundsException =====");

        try {
            int[] numbers = {10, 20, 30};
            System.out.println("Element: " + numbers[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid array index");
        }

        // 3. StringIndexOutOfBoundsException
        System.out.println("\n==== 3. StringIndexOutOfBoundsException =====");

        try {
            String name = "Java";
            System.out.println("Character: " + name.charAt(10));
        } catch (StringIndexOutOfBoundsException e) {
            System.out.println("Invalid String index");
        }

        // 4. NullPointerException
        System.out.println("\n==== 4. NullPointerException =====");

        try {
            String value = null;
            System.out.println("Length: " + value.length());
        } catch (NullPointerException e) {
            System.out.println("Object reference is null");
        }

        // 5. NumberFormatException
        System.out.println("\n==== 5. NumberFormatException =====");

        try {
            String value = "ABC";
            int number = Integer.parseInt(value);
            System.out.println("Number: " + number);
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format");
        }

        // 6. IllegalArgumentException
        System.out.println("\n==== 6. IllegalArgumentException =====");

        try {
            Thread.sleep(-100);
        }
    }
}

```

```

    } catch (IllegalArgumentException e) {
        System.out.println("Invalid argument");
    } catch (InterruptedException e) {
        System.out.println("Thread interrupted");
    }
}

// 7. ClassCastException
System.out.println("\n===== 7. ClassCastException =====");

try {
    Object value = "Java";
    Integer number = (Integer) value;
    System.out.println("Number: " + number);
} catch (ClassCastException e) {
    System.out.println("Invalid type casting");
}

// 8. InputMismatchException
System.out.println("\n===== 8. InputMismatchException =====");

try {
    Scanner sc = new Scanner("ABC");
    int age = sc.nextInt();
    System.out.println("Age: " + age);
    sc.close();
} catch (InputMismatchException e) {
    System.out.println("Please enter a valid integer");
}

// 9. NegativeArraySizeException
System.out.println("\n===== 9. NegativeArraySizeException =====");

try {
    int size = -5;
    int[] numbers = new int[size];
    System.out.println("Array created");
} catch (NegativeArraySizeException e) {
    System.out.println("Array size cannot be negative");
}

// 10. FileNotFoundException
System.out.println("\n===== 10. FileNotFoundException =====");

try {
    FileReader file = new FileReader("abc.txt");
    System.out.println("File opened successfully");
    file.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found");
} catch (IOException e) {
    System.out.println("Error while closing file");
}

System.out.println("\n===== PROGRAM COMPLETED =====");
}
}

```

21. SAMPLE OUTPUT

```

===== 1. ArithmeticException =====
Cannot divide by zero

===== 2. ArrayIndexOutOfBoundsException =====
Invalid array index

===== 3. StringIndexOutOfBoundsException =====
Invalid String index

===== 4. NullPointerException =====
Object reference is null

===== 5. NumberFormatException =====
Invalid number format

===== 6. IllegalArgumentException =====
Invalid argument

===== 7. ClassCastException =====
Invalid type casting

===== 8. InputMismatchException =====
Please enter a valid integer

```

```

===== 9. NegativeArraySizeException =====
Array size cannot be negative

===== 10. FileNotFoundException =====
File not found

===== PROGRAM COMPLETED =====

```

22. REAL-TIME APPLICATIONS

Exception	Real-Time Example
ArithmeticException	Calculator / financial calculations
ArrayIndexOutOfBoundsException	Array-based data processing
StringIndexOutOfBoundsException	Text processing
NullPointerException	Object/reference handling
NumberFormatException	Converting user input to numbers
IllegalArgumentException	Validating method parameters
ClassCastException	Object type conversion
InputMismatchException	Validating user input
NegativeArraySizeException	Dynamic array creation
FileNotFoundException	Reading configuration/data files

23. RESULT

Thus, ten different Java exception classes were studied and successfully demonstrated using suitable Java code examples. The exceptions were handled using appropriate try-catch blocks, allowing the program to continue execution without abnormal termination.